CS102 - Spring 2025/2026

Instructor: Aynur Dayanık
Assistant: Sinan Çavdar

Section No: **2A**

~ Bil-Fit ~

MESMERINEERS

Group Members

Name	Surname
Berkin İlkan	Seçkin
Onur Arda	Özçimen
Berke	Özkan
Samet Talha	Zorlu
Göktan	Arslan

Detailed Design Report

Due: 15 March 2026

1. Introduction

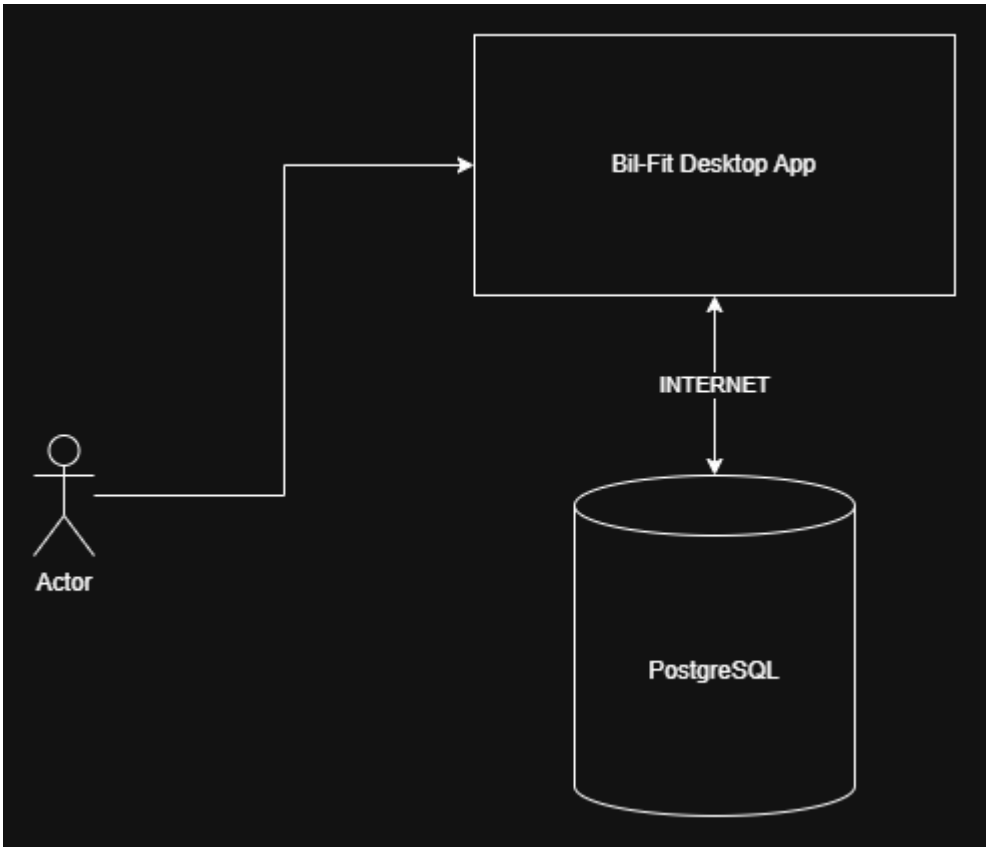
Our app, Bil-Fit (Bilkent Sports Management and Fitness System) aims to be an all-in-one app for sport activities at Bilkent University. Key features of our app are sports hall reservation, viewing and joining tournaments as individuals or teams, dueling 1-1 with other people and letting them see other people's scores for a fair match for students and creating and changing details of tournaments, entering available hours for facilities for reservation and banning low score users temporarily for admins.

2. Details

2.1. System Overview

Bil-Fit is designed as a desktop-based application. Users will run the application on their personal computer. The application will be connected with a database via the internet in order to save and read the information such as personal information, tournaments, and reservations.

2.1.1 Architecture Diagram

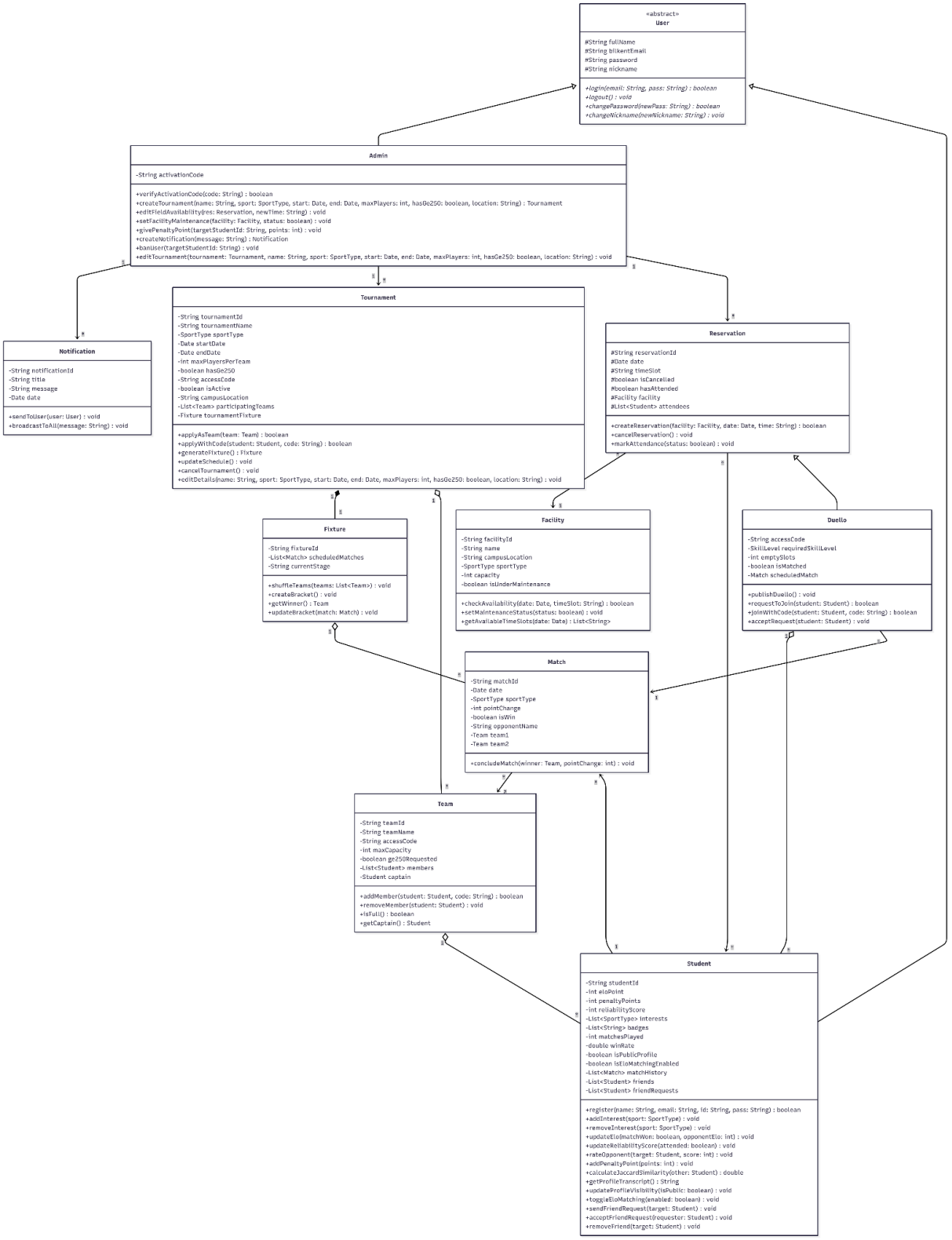


2.1.2 Technologies Used

- **Language**: Java will be used as the primary programming language.
- **GUI**: JavaFX will be utilized for the graphical user interface.
- **Database**: A PostgreSQL cloud server provided by Supabase will be used. A relational database like SQL provides a standardized language and data integrity.
- **Version Control**: Git and Github will be used to track progress and manage the project.

2.2. Core Design Details

2.2.1. UML Class Diagram



UML Diagram Link : https://ibb.co/PkTxwg3

2.2.2. Class Descriptions

2.2.2.1. User Hierarchy (User, Admin, Student)

User: This is an abstract base class. It holds the basic system details and login features for every account type. We put these common features into an abstract base class because it is a basic rule of object oriented design (Booch et al., 2007).

- **Attributes:**
  - **fullName**: Stores the real name of the user.
  - **bilkentEmail**: Acts as the primary identifier and contact address for Bilkent students and staff.
  - **nickname**: An optional username displayed to others in the system.
  - **password**: Protects the account from unauthorized entries.
- **Methods:**
  - **login(email, pass)**: Verifies the entered details to open the user's session.
  - **logout( )**: Safely closes the active session.
  - **changePassword(newPass)**: Changes the old password with a new one.
  - **changeNickname(newNickname)**: Allows to update their nickname.

Student: This class extends the base User class. It keeps important game rules, performance details, and privacy settings. We also added lists for friends, friendRequests, and a matchHistory to improve the social and tracking aspects of the application.

- **Attributes:**
  - **studentId**: The unique school ID of the student.
  - **eloPoint**: A numerical value used to match players with similar skill levels.
  - **penaltyPoints**: Points given when a student does not show up for a match.
  - **reliabilityScore**: A percentage that aims to show the student's dependability for scheduled matches.
  - **interests**: A list for storing which sports the user likes playing (**List<SportType>**).
  - **badges**: Keeps track of the special rewards a student collects (**List<String>**).
  - **matchesPlayed**: A counter that shows the total number of games played.
  - **winRate**: A double value that shows the success percentage of the student.
  - **isPublicProfile**: A boolean to determine if other users can view this profile.
  - **isEloMatchingEnabled**: A boolean to check their competitive choice for matchmaking.
  - **matchHistory**: Stores the past matches of the student (**List<Match>**).
  - **friends**: A list containing the accepted social connections (**List<Student>**).
  - **friendRequests**: A list holding pending connection requests (**List<Student>**).

- **Methods:**
    - **register(name, email, id, pass)**: Opens a fresh profile for a new student.
    - **addInterest(sport)** and **removeInterest(sport)**: Users use these to modify(add or drop) their sport preferences.
    - **updateElo(matchWon, opponentElo)**: Recalculates and update the skill points based on the match outcome.
    - **updateReliabilityScore(attended)**: Modifies the user's dependability metric according to their match presence.
    - **rateOpponent(target, score)**: Allows users to evaluate their opponents after a match.
    - **addPenaltyPoint(points)**: The system uses this to punish students if they miss their reservations.
    - **calculateJaccardSimilarity(other)**: Looks at a user's interests list and compares it with other students to find good sports partners.
    - **getProfileTranscript( )**: Directly creates and returns a short string summary of the user's past games.
    - **updateProfileVisibility(isPublic)**: They can also hide or show their profiles using this.
    - **toggleEloMatching(enabled)**: Users can turn their competitive tracking on or off with this.
    - **sendFriendRequest(target)**, **acceptFriendRequest(requester)**, **removeFriend(target)**: Furthermore, students can now manage their social connections using these.

Admin: This also inherits from the User class. It represents the university staff and sports coordinators. They have special powers in the system.

- **Attributes:**
    - **activationCode**: A special string attribute to verify real staff members during registration.
- **Methods:**
    - **verifyActivationCode(code)**: For security reasons, creating an admin account always needs this method.
    - **createTournament(...)** and **editTournament(...)**: Admins can organize events using these. Instead of relying on external data classes, these methods now take the event details directly as parameters to streamline the process.
    - **editFieldAvailability(res, newTime)** and **setFacilityMaintenance(facility, status)**: Admins use these to manage the daily tasks and sports fields.
    - **givePenaltyPoint(targetStudentId, points)** and **banUser(targetStudentId)**: If students behave badly, admins can punish them using these methods.
    - **createNotification(message)**: They send updates to users with this method.

2.2.2.2. Tournament Management (Tournament, Team, Fixture, Match, Duello)

Tournament: This represents a real sports event organized by an admin. It takes the physical concept of a competition and puts it into our code (Evans, 2003).

- **Attributes:**
  - **tournamentId**: A specific ID number for that exact competition.
  - **tournamentName**: What the event is called on the screen.
  - **sportType**: Shows the specific sport category for the event.
  - **startDate** and **endDate**: Specifies the exact timeframe of the competition.
  - **maxPlayersPerTeam**: Defines the maximum player count for a participating group.
  - **hasGe250**: A boolean indicating if the tournament provides GE250 points.
  - **accessCode**: A special code for private tournament entries.
  - **isActive**: A status boolean showing if the event is currently running.
  - **campusLocation**: Specifies where the matches will take place.
  - **participatingTeams**: Directly holds the list of teams in the tournament (**List<Team>**).
  - **tournamentFixture**: Holds the generated match bracket (**Fixture**).
- **Methods:**
  - **applyAsTeam(team)**: Team captains can register their whole team with this.
  - **applyWithCode(student, code)**: Single players can join quickly using this.
  - **generateFixture( )**: Creates the game schedule.
  - **updateSchedule( )**: Modifies the dates or times of the ongoing fixture.
  - **cancelTournament( )**: Stops the event completely.
  - **editDetails(...)**: Helps admins change the main settings of the tournament.

Fixture: This class manages how the tournament moves forward.

- **Attributes:**
  - **fixtureId**: The unique ID of the bracket system.
  - **scheduledMatches**: Keeps track of all planned games (**List<Match>**).
  - **currentStage**: A string showing the current round (like Semi Finals).
- **Methods:**
  - **shuffleTeams(teams)** and **createBracket( )**: These methods make fair match plans automatically. This saves admins from doing it by hand.
  - **updateBracket(match)**: Moves the winning teams to the next round based on match results.
  - **getWinner( )**: Finds and returns the final champion team.

Team: This class represents a group of Student objects.

- **Attributes:**
  - **teamId**: Unique identifier for the team.
  - **teamName**: The display name chosen by the players.
  - **accessCode**: A password required to join the team.
  - **maxCapacity**: The maximum number of students allowed.
  - **ge250Requested**: A boolean to check if the team wants GE250 credits.
  - **members**: Stores the student objects in a list (**List<Student>**).
  - **captain**: Designates the team leader (**Student**).
- **Methods:**
  - **addMember(student, code)**: Adds players safely.
  - **removeMember(student)**: Takes players out of the team.
  - **isFull( )**: Checks if the team has reached its player limit.
  - **getCaptain( )**: Finds out who the team leader is.

Match: This represents just one game in the system.

- **Attributes:**
  - **matchId**: Unique identifier for the match.
  - **date**: The exact time and day the match happens.
  - **sportType**: The type of sport being played.
  - **pointChange**: How much the ELO points will change.
  - **isWin**: A boolean to show if the user won.
  - **opponentName**: The name of the opposing side.
  - **team1** and **team2**: Directly links to the competing teams to clarify who is playing (**Team**).
- **Methods:**
  - **concludeMatch(winner, pointChange)**: Finishes the game, sets the winning team, and updates the scores.

Duello: This extends the Reservation class. It is a competitive type of booking where users can find opponents for individual sports.

- **Attributes:**
  - **accessCode**: Holds a code for private games.
  - **requiredSkillLevel**: Checks the preferred skill range of the players.
  - **emptySlots**: Tracks how many players are still needed.
  - **isMatched**: A boolean showing if the match is ready.
  - **scheduledMatch**: Holds the planned match details directly (**Match**).

- **Methods:**
    - **publishDuello( )**: A user can share a challenge with everyone using this.
    - **requestToJoin(student)**: Other players can ask to play by using this.
    - **acceptRequest(student)**: The creator can say yes with this.
    - **joinWithCode(student, code)**: Also friends can join private games directly with this.

2.2.2.3. Facility & Reservations (Facility, Reservation)

Facility: This class represents the real sports fields at the university.

- **Attributes:**
    - **facilityId**: Unique ID for the sports area.
    - **name**: The display name of the field (e.g., Main Sports Hall).
    - **campusLocation**: The physical location of the facility.
    - **sportType**: The type of sport this facility is built for.
    - **capacity**: Maximum number of people allowed inside.
    - **isUnderMaintenance**: A status that shows if the sport field is open / closed.
- **Methods:**
    - **checkAvailability(date, timeSlot)**: Checks and verifies if the selected hours are free in order to prevent schedule conflicts.
    - **setMaintenanceStatus(status)**: Provides staff to block (avoid) reservations when a field needs repairs.
    - **getAvailableTimeSlots(date)**: Returns the empty hours for the chosen day.

Reservation: This class manages the entire facility scheduling system.

- **Attributes:**
    - **reservationId**: A unique code to track the booking.
    - **date**: Shows the exact day of the booking.
    - **timeSlot**: The specific hour which has been chosen by the user.
    - **isCancelled**: A boolean showing if the booking was revoked.
    - **hasAttended**: A boolean showing if the users actually used the facility.
    - **facility**: Indicates the specific sports ground for the booking (**Facility**).
    - **attendees**: Stores the list of players attending the matches (**List<Student>**).
- **Methods:**
    - **createReservation(facility, date, time)**: Allocates an empty field for the user at a given time.
    - **cancelReservation( )**: Opens up the reserved slot if the players change their plans.
    - **markAttendance(status)**: Checks if the students really came to the field. This directly affects their reliabilityScore.

2.2.2.4. Utility & Communication (Notification)

Notification: This is a helper class. It alerts people when something important happens.

- **Attributes:**
  - **notificationId**: A special ID given to each specific alert.
  - **title**: The main heading of the message sent.
  - **message**: The main text content.
  - **date**: When the notification was created.
- **Methods:**
  - **sendToUser(user)**: Sends a message to one specific person (like a direct message).
  - **broadcastToAll(message)**: Sends an general or specific announcement to everyone in the system (Gamma et al., 1994).

2.2.3 Controller and View Classes:

The app follows a **dashboard-based** layout where the sidebar provides navigation between different system views. The **content area** loads the selected view using a **StackPane** container. Each module is implemented as a separate **view-controller pair** following the **Model–View–Controller** structure. **Here are the Classes:**

LoginView

Displays the login options between Student Login and Admin Login.

LoginController

Manages the navigation in the login screen.

+deployStudentLogin( ): Loads StudentLoginView.

+deployAdminLogin( ): Loads AdminLoginView.

AdminLoginView

Displays the Admin login screen.

AdminLoginController

Handles Authentication of Admins.

+checkValidity( ): Checks if the information given is valid for an Admin account, If it validates it, deploys AdminMainView.

+deployAdminRegister: Redirects to the AdminRegisterView.

+goBack( ): Redirects toLoginView.

AdminRegisterView: Displays the Admin Registration Screen.

AdminRegisterController

Manages the Admin Registration sequence.

-TextField nameField

-TextField emailField

-TextField passwordField

+checkValidity( ): Checks if the information given is valid for an Admin account, If it validates it, deploys ActivationCodeView.

+goBack( ): Redirects toLoginView**.**

ActivationCodeView:

Displays the activation code screen.

ActivationCodeController

+checkActivationCode: Checks if the given code is valid from the database. If it is, Deploy AdminLoginView.

+goBack: Redirects to AdminLoginView.

StudentLoginView:

Displays the Student login screen.

StudentLoginController

This controller manages the sequence of a Student login screen.

-TextField emailField

-TextField passwordField

+LogIn( ): Validates Info through the model.

+deployHomepage( ): redirects to homepage.

+deployStudentRegister( ): Redirects to StudentRegisterView.

+goBack( ): Redirects to LoginView.

StudentRegisterView: Displays the Student Registration screen.

StudentRegisterController

This controller manages the sequence of a Student register screen.

-TextField nameField

-TextField idField

-TextField emailField

-TextField passwordField

+registerNewStudent( ): Calls to model to create a new Instance of Student.

+deployHomePage( ): Redirects to StudentMainView.

+goBack( ): Redirects StudentLoginView.

MainLayoutController

Is an Abstract Class that consists of a sidebar and content view which gets extended by StudentMainController and AdminMainController.

#StackPane contentView : Determines the area where each menu will reside.

+loadContent(String path): Loads the given FXML to contentView.

+loadHomePage( ): Calls loadContent for Homepage View.

+loadAccount( ): Calls loadContent for Account View.

+loadTournaments( ): Calls loadContent for Tournament View.

+loadReservations( ): Calls loadContent for Reservations View.

+loadSettings( ): Calls loadContent for SettingsView.

+loadFriends( ): Calls loadContent for FriendsView.

StudentMainView

Displays the main Layout of the menu, including the sidebar and the content area.

StudentMainController

Controls the navigation between student views. Extends MainLayoutController

+loadElo( ): Calls loadContent for Elo View.

+loadFriends( ): Calls loadController for FriendsView.

StudentSidebarView

Displays the sidebar where each menu is located.

StudentSidebarController

Controls the navigation in the sidebar.

-StudentMainController mainController

+homeClicked( ): Tells MainController to switch to HomeView.

+accountClicked( ): Tells MainController to switch to AccountView.

+tournamentsClicked( ): Tells MainController to switch to TournamentsView.

+reservationsClicked( ): Tells MainController to switch to ReservationsView.

+eloClicked( ): Tells MainController to switch to EloView.

+settingsClicked( ): Tells MainController to switch to SettingsView.

+friendsClicked( ): Tells MainController to switch to FriendsView.

+logOutClicked( ): Redirects to LoginView.

StudentHomeView

Displays the Upcoming Tournaments and Notifications.

StudentHomeController

-ListView tournamentsList

-ListView notificationsList

+ loadUpcomingTournaments( ): Retrieves the list of upcoming tournaments from the database.

+ loadNotifications ( ): Retrieves the list of notifications from the Student model.

+refresh( )

StudentAccountView

Displays the profile information .

StudentAccountController

-Label nameLabel

-Label idLabel

-ListView interestsList

-ListView MatchesPlayedList

-Label MatchesPlayedLabel

-Label WinRateLabel

-Label EloLabel

+refresh( )

+loadAccountInformation( ): Loads the student's account data from the model.

StudentTournamentsView

Shows the tournaments tab.

StudentTournamentsController

Displays the Tournaments menu.

-ListView upcomingTournamentsList

-ListView myTournamentsList

-ListView partnerRequestsList

+loadUpcomingTournaments( ): Loads the upcoming tournaments.

+loadMyTournaments( ): Loads the user's tournaments**.**

+applyForTournament( ): Opens TournamentApplicationView.

+cancelTournamentApplication( ): Cancels the students tournament application.

+showTournamentSchedule( ): Shows the tournament schedule.

TournamentApplication: Displays the popup window to apply to a tournament.

TournamentApplicationController

-Label tournamentName

-Label tournamentDates

-Label maxPlayers

-TextField teamMemberNameField

-TextField teamMemberEmailField

-TextField teamMemberIDField

+addTeamMember( ): Adds a team member to the tournament team list.

+removeTeamMember( ): Removes a team member to the tournament team list.

+submitApplication( ): Sends the team to the model.

+close( ): Closes the popup.

StudentsReservationsView

Displays reservations for sport facilities.

StudentsReservationsController

-ListView reservationsList

-DatePicker reservationDate

-ComboBox fieldSelection

+reserveField( ): contacts model to add a reservation.

+cancelReservation( ): contacts the model to delete the reservation.

+loadReservations( ): Loads all the user's reservations.

StudentEloView

Displays reservations and the duels tab.

StudentEloController

-ListView reservationsList

-ListView duelsList

+loadReservations( ):

+createDuelFromReservation( ): Creates a new duel from the given reservation.

+loadDuels(boolean isElo): Loads ELO dependent / or not duels made by others.

+requestDuel( ): Requests a duel

StudentFriendsView

Displays a list of friends with an add friend option.

StudentFriendsController

Displays current list of friends.

-ListView friendList

+removeFriend( ): Tells the model to remove the student from the friendlist.

+addFriend( ): Shows AddFriendView.

AddFriendView

Shows a textfield with a given name and a send friend request button.

AddFriendController

-TextField friendName

+sendFriendRequest( ): Contacts model to send a friend request notification.

StudentSettingsView

Displays the settings for the user.

StudentSettingsController

-TextField newPasswordField

-TextField newUsername

-CheckBox notificationsToggle

+toggleElo( ): Toggles elo in duels.

+addInterest( ): Opens the AddInterestView.

+removeInterest( ): Removes the interest from the student.

+changeUsername( ): Changes the username to the one given.

+changePassword( ): Changes the password to the one given.

AddInterestView

Displays a list of all interests for the user to add from.

AddInterestController

+addInterest( ): Contacts the model to add the chosen Interest to the Student.

AdminMainController

Same with students (without elo and friends), Extends MainLayoutController.

AdminSidebarView

Displays the sidebar for admins where each menu is located.

AdminSidebarController

Same structure with Student but without Elo and Friends.

AdminSettingsView

Shows the profile of the admin.

AdminSettingsController

-TextField nameField

-TextField passwordField

+changeName()

+changepassword()

AdminTournamentView

Shows all ongoing tournaments and sections to edit them.

AdminTournamentController

-ListView tournamentsList

+editTournament(): Opens TournamentEditView.

+createNewTournament(): Opens CreateTournamentView.

TournamentEditView.

Contains editable properties of a tournament.

TournamentEditController

-TextField newName

-DatePicker newDate

-Spinner newMaxPlayers

+changeName(): Changes the name of t he tournament.

+changeDate(): Changes the date of the tournament.

+changeMaxPlayers(): Changes the max players on a tournament.

+deleteTournament(): Discards the tournament.

CreateTournamentView

Displays the required inputs to create a new tournament.

CreateTournamentView

-ComboBox SportsType

-TextField name

-DatePicker date

-Spinner maxPlayers

+createTournament(): Creates the tournament with given inputs.

+cancel(): Closes the view without creating the Tournament.

AdminReservationsView

Shows all the current reservations and gives the ability to edit them.

AdminReservationsController

-ComboBox SportsType

-ListView reservations

-DatePicker date

+loadReservations(): Loads reservations for the given sport type and date.

+editReservation(): Opens EditReservationView.

EditReservationView

Displays the editable properties of a reservation.

EditReservationController

-DatePicker newDate

-ComboBox newField

+changeDate(): Changes the date of the reservation.

+changeField(): Changes the field of the reservation.

+deleteReservation(): Discards the reservation from the list.

2.2.4. Design Decisions & Concerns:

2.2.4.1 Overall Design:

-Understandable Implementation: The model is designed to be simple and understandable by any team member regardless of their working area.

-Dividing the work accordingly: The classes are organized neatly with each one managing a specific part of the system.

-Clean Code: Relations and methods between classes are clean, with their naming clearly representing what purpose they serve, which is a fundamental practice for maintainable software development (Martin, 2008).

2.2.4.2 User Security:

-Verified Admins: To prevent false Admins, Admins are given specific activation code for their accounts.

-Giving Power to the Right People: Only verified Admins are given the set of Admin commands such as creating tournaments, banning users and giving penalties.

-Ensuring the User is a Bilkent Student: Students are logged in with their bilkent e-mail and as such are verified to be a Bilkent student.

-Private Matches Stay Private: Events such as duels, can be locked behind an access code which ensures only selected people can enter.

2.3. Task Assignment Distribution:

Database Administrator & Data Access Specialist:

Berkin İlkan Seçkin & Onur Arda Özçimen: Responsible for database structure.

Assigned Tasks:

- Database Architecture & Schema Design *(İlkan)*
- Implementation of Core Data Access Operations (CRUD) *(İlkan)*
- Development of Complex SQL Queries & Data Retrieval *(İlkan)*
- Database Performance Tuning & Query Optimization *(Onur Arda)*

Logic & Algorithms - Operations & Communication:

Backend Developers:

Onur Arda Özçimen & Samet Talha Zorlu: Responsible for OOP, system logic, and algorithms displayed in UML diagrams.

Assigned Class and Features:

- User Hierarchy (User, Student, Admin) *(Onur Arda)*
- Tournament Management (Tournament, Team, Fixture, Match, Duello) *(Samet Talha)*
- Facility & Reservations (Facility, Reservation) *(Samet Talha)*
- Utility & Communication (Notification) *(Onur Arda)*
- MVC Design *(Samet Talha & Onur Arda)*

Student Interface Page - Admin & Management Console:

Frontend Developers:

Berke Özkan & Göktan Arslan: Responsible for designing the JavaFX user interface and connecting user actions to backend structures.

Assigned Class and Features:

- Student Interface Pages (User Dashboard, Profile Page, Matchmaking/Duello Screen) *(Berke)*
- Admin & Management Console (Tournament Creation Page, Facility Status Screen) *(Göktan)*
- GUI Controllers and event listeners binding the visual elements to the core system. *(Berke & Göktan)*

3. Conclusion

In conclusion, Detailed Design Report presents the architectural design and object-oriented structure of Bil-Fit application. The relationships between classes and their fields and methods is displayed in a clear way in the UML diagram. Furthermore, this document reports that tasks were distributed so that each team member had a sufficient workload. Ultimately, our team is ready to develop the project with this foundation.

References

Booch, G., Maksimchuk, R. A., Engle, M. W., Young, B. J., Conallen, J., & Houston, K. (2007). *Object-oriented analysis and design with applications* (3rd ed.). Addison-Wesley Professional.

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley Professional.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Martin, R. C. (2008). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall.